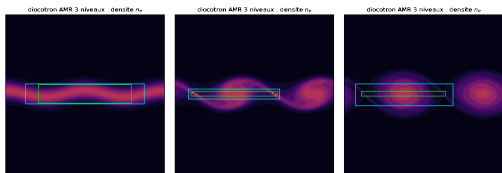


adc_cpp : architecture d'un solveur hyperbolique–elliptique AMR

Abstraction C++, solveurs, AMR, MPI/GPU, résultats ROMEO

$$\frac{\partial U}{\partial t} + \nabla \cdot F(U, \text{aux}) = S(U, \text{aux}), \quad \mathcal{D}\phi = f(U), \quad \text{aux} = (\phi, \nabla\phi)$$



$$\partial_t U + \nabla \cdot F(U, \text{aux}) = S(U, \text{aux}), \quad \mathcal{D}\phi = f(U), \quad \text{aux} = (\phi, \nabla\phi)$$

Partie hyperbolique

- volumes finis cartésiens ;
- flux numérique Rusanov, HLL, HLLC ;
- reconstruction NoSlope, MUSCL, WENO5-Z ;
- intégration SSP-RK / IMEX / splitting.

Partie elliptique

- multigrille géométrique 5 points ;
- FFT périodique mono-rang ou distribuée ;
- champ auxiliaire dérivé par différences centrées ;
- couplage par étage ou une fois par pas.

Source : `include/adc/operator/spatial_operator.hpp`, `include/adc/elliptic/`

Ce que le contrat couvre

- systèmes conservatifs ou quasi conservatifs ;
- flux et sources locaux, évaluables cellule par cellule ;
- fermeture elliptique de type Poisson ou opérateur analogue ;
- champ auxiliaire utilisé dans le flux ou dans la source.

Exemples déjà présents

- diocotron : $\text{aux} \rightarrow F$;
- Euler–Poisson : $\text{aux} \rightarrow S$;
- deux-fluides AP : source raide traitée par intégrateur dédié.

Limite explicite

Un modèle cinétique, non local ou multi-elliptique ne rentre pas seulement par `PhysicalModel`. Il faut alors introduire un opérateur discret plus riche.

Contrat C++ : PhysicalModel

```
template <class M>
concept PhysicalModel =
    requires(const M m, const typename M::State u,
             const typename M::Aux a, int dir) {
        typename M::State;
        typename M::Aux;
        { M::n_vars } -> std::convertible_to<int>;
        { m.flux(u, a, dir) } -> std::same_as<typename M::State>;
        { m.max_wave_speed(u, a, dir) } -> std::convertible_to<Real>;
        { m.source(u, a) } -> std::same_as<typename M::State>;
        { m.elliptic_rhs(u) } -> std::convertible_to<Real>;
    };
```

Source : `include/adc/core/physical_model.hpp`

- Le modèle physique fournit seulement des formules ponctuelles.
- Il ne voit ni MultiFab, ni MPI, ni AMR, ni Kokkos.
- Les types `State` et `Aux` sont compatibles `host/device`.

Exemple réel : modèle diocotron

```
struct Diocotron {
    using State = StateVec<1>;
    using Aux = adc::Aux;
    static constexpr int n_vars = 1;

    Real B0 = 1.0;
    Real n_i0 = 1.0;
    Real alpha = 1.0;

    ADC_HD Real drift_velocity(const Aux& a, int dir) const {
        return (dir == 0) ? (-a.grad_y / B0) : (a.grad_x / B0);
    }
    ADC_HD State flux(const State& u, const Aux& a, int dir) const {
        State f{};
        f[0] = u[0] * drift_velocity(a, dir);
        return f;
    }
    ADC_HD Real max_wave_speed(const State&, const Aux& a, int dir) const {
        const Real d = drift_velocity(a, dir);
        return d < 0 ? -d : d;
    }
    ADC_HD State source(const State&, const Aux&) const { return State{}; }
    ADC_HD Real elliptic_rhs(const State& u) const {
        return alpha * (u[0] - n_i0);
    }
};
```

Source : include/adc/model/diocotron.hpp

Deux usages de aux

Diocotron : aux dans le flux

$$\partial_t n_e + \nabla \cdot (n_e \mathbf{v}_E) = 0, \quad \mathbf{v}_E = \frac{1}{B_0} \begin{pmatrix} -\partial_y \phi \\ \partial_x \phi \end{pmatrix}$$

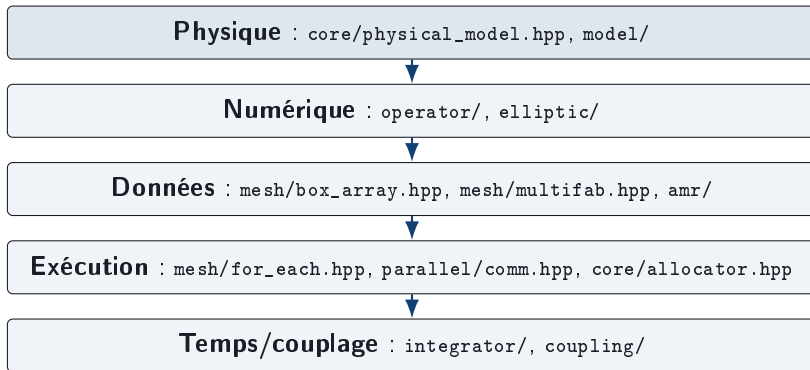
$$\Delta \phi = \alpha (n_e - n_{i0})$$

Euler–Poisson : aux dans la source

$$\begin{aligned} \partial_t U + \nabla \cdot F_{\text{Euler}}(U) &= S(U, \nabla \phi) \\ S &= (0, \rho g_x, \rho g_y, \rho \mathbf{u} \cdot \mathbf{g}), \quad \mathbf{g} = -\nabla \phi \end{aligned}$$

Un même `assemble_rhs` assemble $R = -\nabla \cdot F + S$; seul le modèle change.

Architecture du dépôt



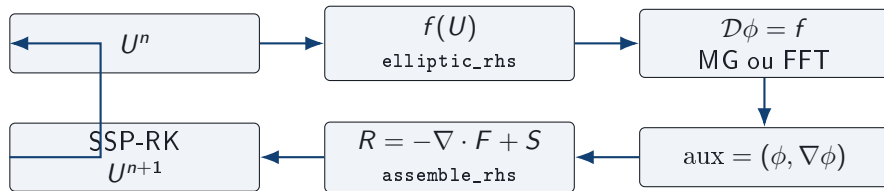
Point architectural

Les conteneurs décrivent où sont les données. Les frontières d'exécution décrivent comment on parcourt ou communique. Les deux ne sont pas mélangés.

Carte des modules principaux

Module	Rôle dans le solveur
core/	Real, ADC_HD, StateVec, Aux, concept PhysicalModel.
model/	Diocotron, Euler, Euler-Poisson, deux-fluides isotherme, Langmuir.
operator/	Reconstructions, flux numériques, assemble_rhs, flux de face pour reflux.
elliptic/	EllipticSolver, multigrille géométrique, FFT Poisson.
mesh/	Box2D, BoxArray, DistributionMapping, Fab2D, MultiFab.
integrator/	SSP-RK, IMEX, splitting, moteur AMR advance_amr.
coupling/	Coupler, AmrCouplerMP, regrid, diagnostics AMR.
solver/, src/	façades compilées libadc pour applications et bindings Python.

Boucle de couplage hyperbolique–elliptique



- `PerStageCoupling` : Poisson résolu à chaque étage RK.
- `OncePerStepCoupling` : champ gelé pendant le pas, plus rapide mais ordre de couplage plus faible.

Code réel : Coupler::advance

```
template <class Limiter = NoSlope, class Policy = PerStageCoupling>
void advance(MultiFab& U, Real dt) {
    MultiFab R(ba_, dm_, Model::n_vars, 0);

    update_aux(U);                      // Poisson -> phi -> aux
    fill_ghosts(U, geom_.domain, bcU_);
    assemble_rhs<Limiter>(model_, U, aux_, geom_, R);
    MultiFab U1 = U;
    saxpy(U1, dt, R);

    if constexpr (std::is_same_v<Policy, PerStageCoupling>)
        update_aux(U1);                // champ recalcule au 2e stage
    fill_ghosts(U1, geom_.domain, bcU_);
    assemble_rhs<Limiter>(model_, U1, aux_, geom_, R);
    saxpy(U1, dt, R);
    lincomb(U, Real(0.5), U, Real(0.5), U1);
}
```

Source : include/adc/coupling/coupler.hpp

Flux de face

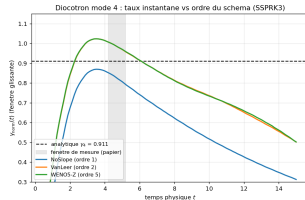
$$\hat{F} = \frac{1}{2}(F(U_L) + F(U_R)) - \frac{1}{2}\alpha(U_R - U_L)$$

$$\alpha = \max \lambda(U_L, U_R, \text{aux})$$

- Rusanov par défaut ;
- HLL/HLLC pour Euler ;
- flux de face conservés pour le reflux AMR.

Reconstruction

- NoSlope : ordre 1, robuste ;
- Minmod, VanLeer, MC : MUSCL ordre 2 ;
- WENO5Z : ordre 5 pour convergence diocotron.



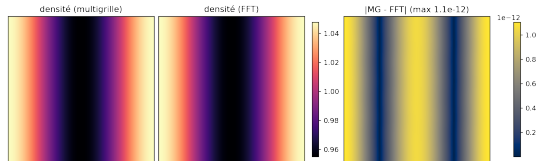
EllipticSolver

- `rhs()` : second membre $f(U)$;
- `phi()` : solution conservée en warm start ;
- `solve()` : inversion ;
- `residual()` : contrôle numérique.

Source : `include/adc/elliptic/elliptic_solver.hpp`

Backends présents

- `GeometricMG` : V-cycle Gauss–Seidel rouge/noir, compatible AMR.
- `PoissonFFTSolver` : direct, périodique, mono-rang.
- `DistributedFFTSolver` : bandes MPI, `MPI_Alltoall`.



Solveurs physiques implémentés

Solveur	Équation	Validation
Diocotron	transport $E \times B$, Poisson sur $n_e - n_{i0}$	croissance linéaire ; invariants ; AMR.
Euler	Euler compressible 2D	Sod ; vortex isentropique.
EulerPoisson	Euler + force $-\nabla\phi$, signe gravité/plasma	Jeans ; Bohm–Gross.
TwoFluidAP	deux-fluides isotherme asymptotic-preserving	limite raide ; dispersion ; cyclotron.

Frontière bibliothèque

`solver/` et `src/` fournissent des façades compilées non templatisées. Le coeur générique reste dans `include/adc/`.

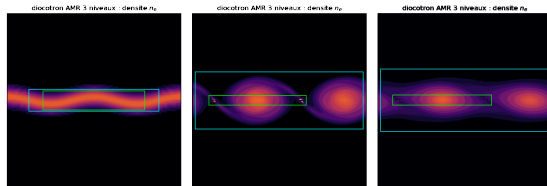
AMR block-structured

Objets

- `BoxArray` : liste globale des patches ;
- `DistributionMapping` : `patch` $\rightarrow$ rang ;
- `MultiFab` : champ distribué par patches ;
- `AmrLevelMP` : U , aux, Δx , Δy .

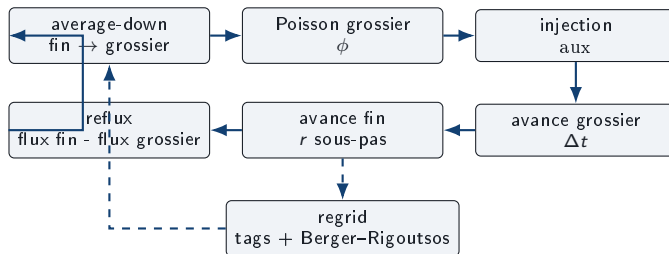
Invariants

- ratio 2 entre niveaux ;
- sous-cyclage temporel Berger–Oliger ;
- reflux aux interfaces coarse–fine.



Diocotron AMR 3 niveaux, frames extraites de `docs/anim_diocotron_amr3.gif`.

Pas AMR conservatif



- `compute_face_fluxes` expose les flux nécessaires au reflux.
- `CoverageMask` évite de corriger les interfaces fin-fin.
- `FluxRegister` agrège average-down et reflux, y compris sous MPI.

Code réel : entrée AMR de production

```
void step(Real dt) {  
    update(); // sync_down + Poisson + grad(phi) + injection aux  
    advance_amr<NoSlope, RusanovFlux>(  
        model_, stack_.L(), stack_.domain(), dt,  
        Periodicity{true, true}, replicated_coarse_);  
}  
  
template <class Crit>  
void regrid(Crit crit, int grow = 2, int margin = 2) {  
    amr_regrid_finetest(stack_.L(), stack_.aux(),  
        stack_.domain(), crit, grow, margin);  
}
```

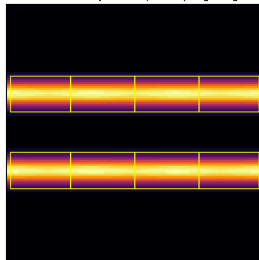
Source : include/adc/coupling/amr_coupler_mp.hpp

AmrCouplerMP orchestre. Le moteur conservatif est `advance_amr` ; le remaillage est isolé dans `amr_regrid_finetest`.

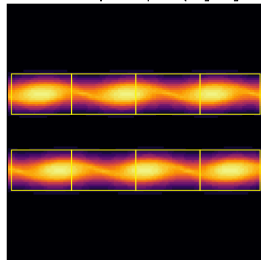
Pipeline

- 1 tag_cells : critère fourni par le solveur utilisateur ;
- 2 grow_tags : buffer pour anticiper le déplacement ;
- 3 berger_rigoutsos : regroupement en patches ;
- 4 interpolation depuis le parent ;
- 5 report de l'ancien fin là où il existe.

flucotron AMR multi-patch : 8 patches (Berger-Rigoutsos)



flucotron AMR multi-patch : 8 patches (Berger-Rigoutsos)



Critère

Le critère de raffinement est physique.
L'infrastructure AMR est générique.

Frontière d'exécution : for_each_cell

```
template <class F>
void for_each_cell(const Box2D& b, F f) {
#ifdef ADC_HAS_KOKKOS
    Kokkos::parallel_for(
        "adc_for_each_cell",
        Kokkos::MDRangePolicy<Kokkos::Rank<2>, Kokkos::IndexType<int>>(
            {b.lo[0], b.lo[1]}, {b.hi[0] + 1, b.hi[1] + 1}),
        f);
#elif defined(_OPENMP)
    const long n_cells = long(b.hi[0] - b.lo[0] + 1) *
                        long(b.hi[1] - b.lo[1] + 1);
#pragma omp parallel for collapse(2) schedule(static) if (n_cells >= 4096)
    for (int j = b.lo[1]; j <= b.hi[1]; ++j)
        for (int i = b.lo[0]; i <= b.hi[0]; ++i) f(i, j);
#else
    for (int j = b.lo[1]; j <= b.hi[1]; ++j)
        for (int i = b.lo[0]; i <= b.hi[0]; ++i) f(i, j);
#endif
}
```

Source : include/adc/mesh/for_each.hpp

Structures

- `DistributionMapping(nboxes, nrank)` : round-robin par défaut ;
- `parallel_copy` : copie inter-patches et inter-rangs ;
- `fill_boundary` : halos intra-niveau ;
- `all_reduce_*` : diagnostics et registres AMR.

Tests MPI : chemins AMR multi-patch bit-identiques pour $np = 1, 2, 4$ sur les tests dédiés.

Politique niveau 0

`AmrCouplerMP` supporte un grossier répliqué par rang, ou un grossier multi-box distribué. Le répliqué reste le défaut pour les cas ROMEO testés.

Source : `include/adc/parallel/comm.hpp`

GPU : Kokkos/CUDA sans réécrire les opérateurs

Conditions dans le code

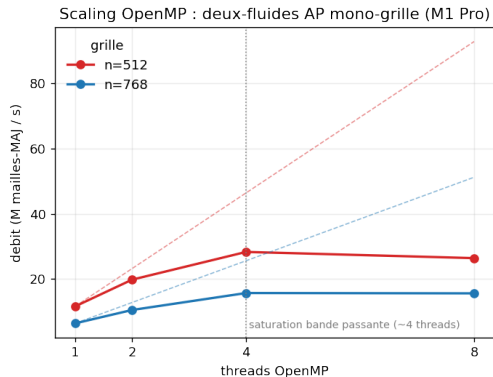
- lambdas ADC_HD ;
- captures POD : Array4, ConstArray4, scalaires ;
- pas de polymorphisme dynamique dans les kernels ;
- device_fence() avant lecture hôte.

CMake

```
cmake -B build-gpu -DADC_USE_KOKKOS=ON \  
-DCMAKE_CXX_COMPILER=$K/bin/nvcc_wrapper \  
-DKokkos_ROOT=$K
```

Exemples

- examples/gpu/diocotron_amr_kokkos.cpp
- examples/gpu/diocotron_amr_mpi_kokkos.cpp



Observations mesurées

- Pas Euler–Poisson : temps dominé par Poisson.
- `OncePerStepCoupling` : 52.3 → 19.8 ms/pas sur M1 Pro.
- FFT périodique : 76 → 16 ms/pas sur Euler–Poisson.
- OpenMP utile seulement si le grain est suffisant ; sinon le coût de fork/join domine.

Source : docs/PERFORMANCE.md

Étape 1 : écrire le modèle physique

```
struct MyModel {  
    using State = adc::StateVec<N>;  
    using Aux    = adc::Aux;  
    static constexpr int n_vars = N;  
  
    ADC_HD State flux(const State& u, const Aux& a, int dir) const;  
    ADC_HD Real max_wave_speed(const State& u, const Aux& a, int dir) const;  
    ADC_HD State source(const State& u, const Aux& a) const;  
    ADC_HD Real elliptic_rhs(const State& u) const;  
};  
  
static_assert(adc::PhysicalModel<MyModel>);
```

Règles pratiques : fonctions locales, types POD, pas d'allocation ni `std::function` dans les formules appelées en kernel.

Étape 2 : grille uniforme couplée

```
adc::Box2D dom = adc::Box2D::from_extents(n, n);
adc::Geometry geom{dom, 0.0, L, 0.0, L};
adc::BoxArray ba({dom});
adc::DistributionMapping dm(ba.size(), adc::n_ranks());
adc::MultiFab U(ba, dm, MyModel::n_vars, /*ngrow=*/2);

MyModel model{/*parametres*/};
adc::BCRec bcU, bcPhi;
adc::Coupler<MyModel, adc::GeometricMG> sim(model, geom, ba, bcU, bcPhi);

for (int s = 0; s < nsteps; ++s)
    sim.advance<adc::VanLeer>(U, dt);
```

Variante périodique rapide : remplacer `GeometricMG` par un backend FFT conforme à `EllipticSolver`, quand les hypothèses FFT sont vérifiées.

Étape 3 : AMR

```
std::vector<adc::AmrLevelMP> levels;
levels.push_back({std::move(U0), nullptr, dx0, dy0});
levels.push_back({std::move(U1), nullptr, dx0/2, dy0/2});

adc::AmrCouplerMP<MyModel> sim(model, geom, ba0, bc, std::move(levels));

auto crit = [&](const adc::ConstArray4& u, int i, int j) {
    return indicator(u, i, j) > threshold;
};

for (int s = 0; s < nsteps; ++s) {
    if (s % 20 == 0) sim.regrid(crit);
    sim.step(dt);      // sync_down + Poisson + injection aux + advance_amr
}
```

L'utilisateur choisit le critère. ADC fournit clustering, nesting, interpolation, sous-cyclage, average-down et reflux.

Étape 4 : MPI et GPU

MPI

```
cmake -B build-mpi -DADC_USE_MPI=ON
cmake --build build-mpi
mpirun -np 4 ./build-mpi/bin/diocotron_mpi out 512 600
```

- appeler `comm_init` au début du `main` ;
- distribuer par `DistributionMapping` ;
- garder les diagnostics via `all_reduce_*`.

GPU

```
cmake -B build-gpu -DADC_USE_KOKKOS=ON \
  -DCMAKE_CXX_COMPILER=$K/bin/nvcc_wrapper \
  -DKokkos_ROOT=$K
cmake --build build-gpu
```

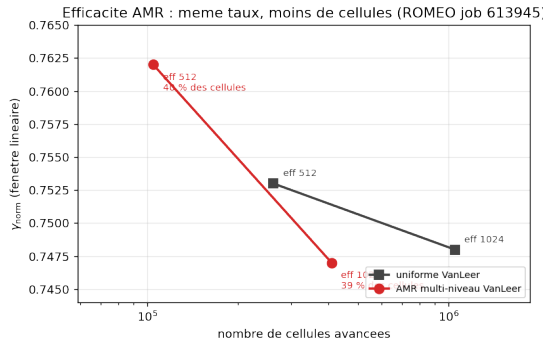
Le modèle ne change pas. Les fonctions appelées dans les boucles doivent rester `ADC_HD` et `device-safe`.

ROMEO : exécutions réalisées

Job	Configuration	Résultat exploitable
613961	WENO5-Z + SSPRK3, modes 3, 4, 5, eff. 256–1024	runs stables ; croissance mesurée, $R^2 = 1.00$.
613945	colonne diocotron, NoSlope/-VanLeer/AMR, eff. 512–1024	AMR VanLeer suit l'uniforme avec environ 40% des cellules.
614089	GH200, Kokkos/CUDA, compute-sanitizer	memcheck/initcheck/synccheck sans erreur ; checksum CPU/GPU identique.
614125	convergence $l = 3, 4, 5$, fenêtre linéaire [3, 9]	mode 3 exact, mode 4 proche, mode 5 biaisé : effet géométrique probable.

Source : romeo/HERO_RESULTS.md, romeo/CONV_RESULTS.md

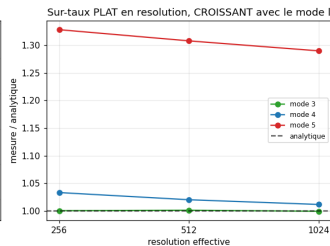
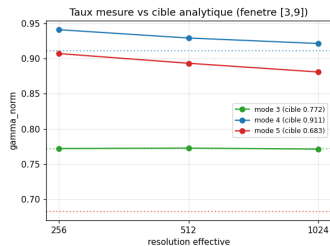
ROMEO : AMR vs uniforme



Cas	Cellules	γ_{lin}
uniforme VanLeer 512	262144	0.753
AMR VanLeer 512	104632	0.762
uniforme VanLeer 1024	1048576	0.748
AMR VanLeer 1024	409008	0.747

Résultat : l'AMR reproduit le taux uniforme à résolution effective identique, avec moins de la moitié des cellules.

ROMEO : taux de croissance diocotron

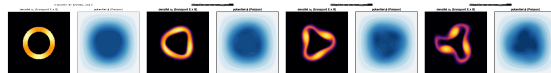


Mode	cible	eff. 1024
$l = 3$	0.772	0.771
$l = 4$	0.911	0.921
$l = 5$	0.683	0.881

Le biais ne décroît pas avec le raffinement intérieur. L'hypothèse actuelle est une erreur géométrique sur la représentation cartésienne de l'anneau, plus marquée aux hauts modes.

Job 614089, GH200

- build Kokkos/CUDA validé ;
- coupled_kokkos : memcheck, initcheck, synccheck sans erreur ;
- diocotron_amr_kokkos : memcheck sans erreur ;
- checksum 4394594.404318, identique CPU/GPU ;
- dérive de masse 2.22×10^{-16} .



Exemple de run diocotron anneau ; le test GPU valide surtout la discipline mémoire et les fences.

Numérique

- cut-cell ou level-set sur les bords r_0, r_1 de l'anneau ;
- comparaison grille polaire pour isoler l'erreur géométrique ;
- extension des opérateurs elliptiques à coefficients variables.

HPC

- load balancing SFC/knapsack pour AMR multi-patch ;
- tagging distribué sur niveau intermédiaire pour AMR 3+ niveaux ;
- choix automatique grossier répliqué vs distribué ;
- séparation éventuelle des cibles `adc_serial`, `adc_mpi`, `adc_kokkos`.

Messages techniques à défendre

- ❶ Le contrat `PhysicalModel` isole la physique des choix AMR/MPI/GPU.
- ❷ `assemble_rhs`, `Coupler` et `advance_amr` forment trois frontières séparées : opérateur spatial, couplage elliptique, hiérarchie AMR.
- ❸ L'AMR est conservatif par flux de face, average-down et reflux coverage-aware.
- ❹ La parallélisation n'est pas injectée dans les modèles : elle passe par `for_each_cell`, `DistributionMapping` et `comm`.
- ❺ Les runs ROMEO valident à la fois la physique diocotron, l'équivalence AMR/uniforme et la portabilité Kokkos/CUDA.